

CSE 3204
Formal Language, Automata and Computability



Lecture 23
Turing Machine

Simulating a Turing Machine by a Computer

- Given a TM **M**, write a program that acts like M.
- Since there are only a finite number of states and a finite number of transition rules, states as encoded as character strings and a table of transitions is used.
- the tape symbols can be encoded as character strings of a fixed length.
- Representation of an infinite tape is performed using two stacks containing a number of disks.

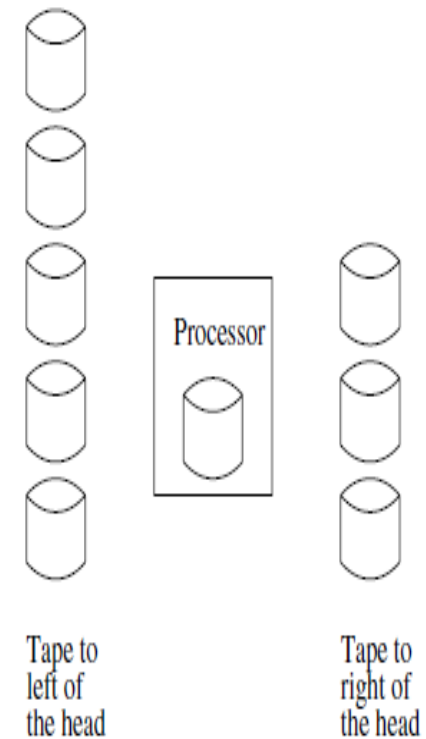


Figure 8.21: Simulating a Turing machine with a common computer

Simulating a Computer using a Turning Machine

- Remaining from the book.

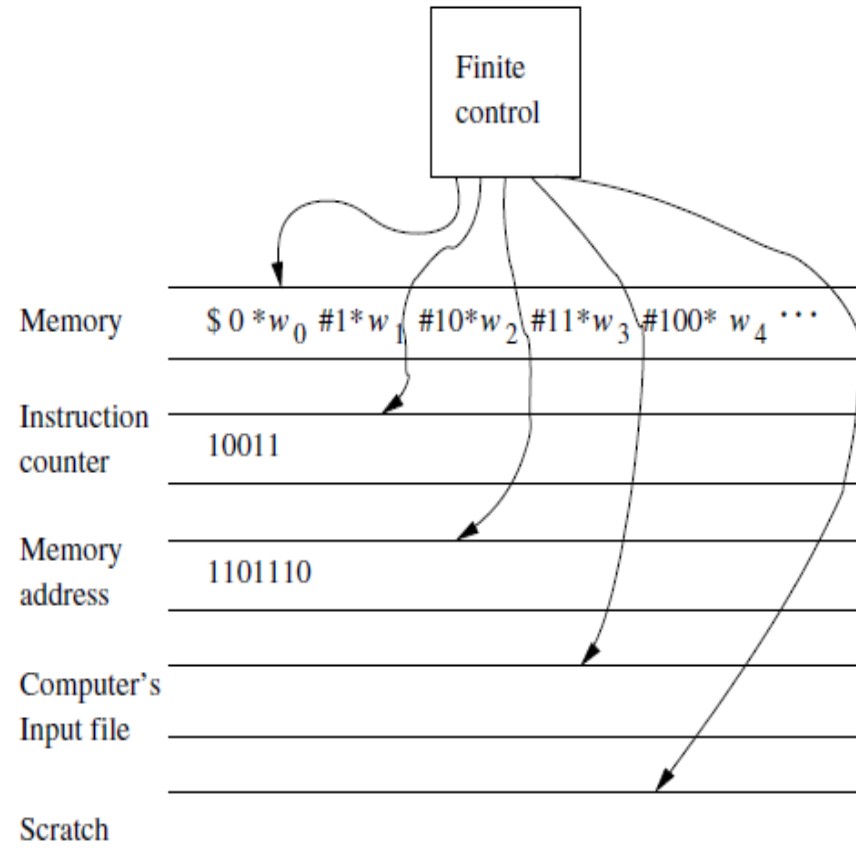


Figure 8.22: A Turing machine that simulates a typical computer

Running Times of Computers and Turing Machines

- The issue of running time is important because we shall use the TM not only to examine the question of what can be computed at all, but what can be computed with enough efficiency that a problem's computer-based solution can be used in practice.
- The dividing line separating the *tractable* — that which can be solved efficiently — from the *intractable* — problems that can be solved, but not fast enough for the solution to be usable — is generally held to be between what can be computed in polynomial time and what requires more than any polynomial running time.
- Thus, we need to assure ourselves that if a problem can be solved in polynomial time on a typical computer, then it can be solved in polynomial time by a Turing machine, and conversely. Because of this polynomial equivalence, our conclusions about what a Turing machine can or cannot do with adequate efficiency apply equally well to a computer.

Theorem 8.10: The time taken by the one-tape TM N of Theorem 8.9 to simulate n moves of the k -tape TM M is $O(n^2)$.

Running Times of Computers and Turing Machines

- No assumption is made on the number of bits that one computer word can hold.
- If a word holding integer 2 is multiplied by the word by itself for n consecutive steps, then the word would hold the number 2^{2^n} .
- This number requires $2^n + 1$ bits to represent.
- The time taken Turing machine to simulate these n instructions would be exponential in n , at least.
- Assumption: the computer may use words that grow to any length, but one computer instruction can only produce a word that is one bit longer than the longer of its arguments.

Running Times of Computers and Turing Machines

Example 8.16: Under the above restriction, addition is allowed, since the result can only be one bit longer than the maximum length of the addends. Multiplication is not allowed, since two m -bit words can have a product of length $2m$. However, we can simulate a multiplication of m -bit integers by a sequence of m additions, interspersed with shifts of the multiplicand one bit left (which is another operation that only increases the length of the word by 1). Thus, we can still multiply arbitrarily long words, but the time taken by the computer is proportional to the square of the length of the operands. $\square$

Assuming one-bit maximum growth per computer instruction executed, we can prove our polynomial relationship between the two running times. The idea of the proof is to notice that after n instructions have been executed, the number of words mentioned on the memory tape of the TM is $O(n)$, and each computer word requires $O(n)$ Turing-machine cells to represent it. Thus, the tape is $O(n^2)$ cells long, and the TM can locate the finite number of words needed by one computer instruction in $O(n^2)$ time.

Theorem 8.17: If a computer:

1. Has only instructions that increase the maximum word length by at most 1, and
2. Has only instructions that a multitape TM can perform on words of length k in $O(k^2)$ steps or less,

then the Turing machine described in Section 8.6.2 can simulate n steps of the computer in $O(n^3)$ of its own steps.

Proof from the book.

Theorem 8.18: A computer of the type described in Theorem 8.17 can be simulated for n steps by a one-tape Turing machine, using at most $O(n^6)$ steps of the Turing machine. $\square$

- Section 8.6 of Chapter 8